

Introduction to NLP — Teaching Machines to Read

What is NLP?

Natural Language Processing — making computers understand the way humans talk and write.

Introduction to Natural Language Processing (NLP)

Think about how naturally you understand this sentence:

"I didn't say she stole the money."

Depending on which word you stress, it can have several different meanings.

- I didn't say she stole the money. → Someone else said it.
- I didn't say she stole the money. → I deny saying it.
- I didn't say she stole the money. → I may have implied it.
- I didn't say she stole the money. → Someone else stole it.
- I didn't say she stole the money. → Maybe she borrowed it.
- I didn't say she stole the money. → Perhaps she stole something else.

Humans understand these subtle differences naturally.

Now imagine teaching that to a computer.

That is exactly the challenge of Natural Language Processing (NLP) and one of the reasons it is one of the most exciting fields in Artificial Intelligence.

=====

Where Do You Use NLP Every Day?

1. Google Search

When you type:

"best pizza near me"

Google understands:

- best → quality preference
- pizza → food category
- near me → location intent

and returns relevant nearby restaurants.

2. Gmail Smart Reply

Suppose you receive:

"Can we meet tomorrow at 10 AM?"

Gmail may suggest:

- Sounds good!
- I'll check.
- See you tomorrow.

The system understands the email's context using NLP.

3. Autocorrect

You type:

teh

Your phone suggests:

the

NLP combined with statistical models predicts the word you intended to type.

4. Alexa, Siri, and Google Assistant

Voice Command:

"Set a timer for 10 minutes."

Process:

1. Speech is converted to text.
2. The text is analyzed.
3. The user's intent is identified.
4. The action is executed.

This entire pipeline relies heavily on NLP.

5. Spam Filters

Email:

"Congratulations! You won a FREE iPhone!"

The system analyzes the message and classifies it as:

Spam

using NLP-based text classification techniques.

6. ChatGPT

ChatGPT can:

- Understand questions
- Track conversation context
- Generate human-like responses
- Answer follow-up questions

All of this is powered by advanced NLP models.

=====

Why Is Human Language Hard for Computers?

Numbers are easy:

$5 + 3 = 8$

The answer is always the same.

Language, however, is messy and full of hidden meanings.

1. Ambiguity

Sentence:

"I saw the man with the telescope."

Possible meanings:

- I used a telescope to see the man.
- The man was holding a telescope.

Humans use context to determine the correct meaning.

Computers often struggle without additional information.

2. Sarcasm

Sentence:

"Oh great, another Monday."

Human interpretation:

Negative emotion

Computer interpretation (without context):

Positive emotion because of the word "great"

Understanding sarcasm remains a difficult NLP problem.

3. Context

Sentence:

"He kicked the bucket."

Literal meaning:

He physically kicked a bucket.

Actual meaning:

He died.

Humans easily understand idioms and expressions through context.

Computers must learn these patterns from large amounts of data.

4. Abbreviations and Slang

Examples:

- LOL = Laugh Out Loud
- BRB = Be Right Back
- GOAT = Greatest Of All Time

Language evolves constantly, making it difficult for NLP systems to stay current.

5. Multiple Languages

Example: chat

English: chat = conversation

French: chat = cat

Same spelling, completely different meaning.

NLP systems must identify the language before interpreting the word correctly.

=====

Key Challenges in NLP

1. Ambiguity

Example: "I saw the man with the telescope"

2. Sarcasm

Example: "Oh great, another Monday"

3. Context

Example: "He kicked the bucket"

4. Slang and Abbreviations

Examples: LOL, BRB, GOAT

5. Multiple Languages

Example: chat (English vs French)

Real-Life Analogy — NLP as a Translator

Imagine you hired a brilliant translator who has read every book ever written but has never spoken to a human.

This translator:

- Understands grammar perfectly.
- Can recognize language patterns.
- Knows the meanings of millions of words.
- Can predict what word is likely to come next.
- Has read enormous amounts of text.

At first glance, this sounds like the perfect language expert.

However, there is one problem:

The translator has never experienced real human emotions, intentions, humor, or sarcasm.

For example:

Person A: "Oh great, another Monday."

A human immediately understands:

The speaker is probably unhappy.

The translator sees:

"great" = positive word

and may incorrectly assume the sentence expresses happiness.

Another example:

Person A: "Can you open the window?"

Humans understand:

This is a polite request.

The translator may interpret it literally as a question about someone's ability.

This is very similar to how modern NLP models work.

NLP models:

- Learn from massive amounts of text.
- Identify patterns in language.
- Understand grammar and sentence structure.
- Predict likely meanings based on context.
- Generate human-like responses.

But they do not truly experience:

- Happiness
- Sadness
- Humor
- Sarcasm
- Personal memories
- Real-world consciousness

They learn statistical patterns rather than human experiences.

The Goal of NLP

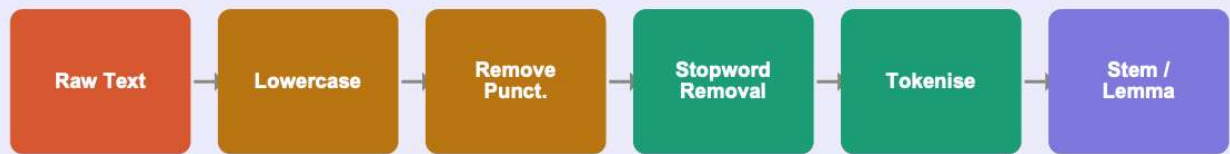
Our job as AI engineers is to give NLP systems the tools needed to bridge the gap between:

Human Language ↓ Understanding Context ↓ Understanding Intent ↓ Generating Meaningful Responses

The better the model understands context, emotions, and intent, the more natural its communication becomes.

✓ Text Pre-processing Pipeline

Clean and normalise raw text before any model ever sees it. Raw text from the internet is messy: random capitalisation, emojis, HTML tags, punctuation everywhere, and filler words like 'the' and 'is' that carry no meaning. Just as you wash vegetables before cooking, you must clean text before feeding it to a model. Here is the standard 5-step pipeline:



✓ Step 1 — Lowercasing

'Hello', 'HELLO', and 'hello' are the same word but a computer sees three different tokens. Converting everything to lowercase collapses them into one. Simple but critical.

```
# Lowercasing — always the first step
text = 'Hello! This is a SAMPLE Email to check Manoj And Ram. Are looking at Screen.'
# Python's built-in .lower() method
text_lower = text.lower()
print(text_lower)
```

```
hello! this is a sample email to check manoj and ram. are looking at screen.
```

✓ Step 2 — Removing punctuation & special characters

Punctuation like '!', '@', '#' usually adds no value for classification. We use regular expressions (regex) to strip them out.

```
import re
text = 'Hello! This is a SAMPLE Email to check Manoj And Ram. Are looking at Screen.'
text_clean = re.sub(r'[^\w\s]', '', text)
print(text_clean)
```

```
ello his is a mail to check anoj nd am re looking at creen
```

✓ Step 3 — Stopword removal

Stopwords are ultra-common words — 'the', 'is', 'a', 'and', 'in' — that appear everywhere but carry no distinctive meaning. Removing them reduces noise and shrinks your vocabulary. NLTK ships with a ready-made stopwords list.

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords', quiet=True)
stop_words = set(stopwords.words('english'))
text = "I'm going to College today"
tokens = []
for word in text.split():
    if word not in stop_words:
        tokens.append(word)
print(tokens)
```

```
["I'm", 'going', 'College', 'today']
```

✓ Step 4 — Tokenisation

Tokenisation means splitting a sentence into its individual units (tokens). A simple `.split()` works for most cases, but NLTK's `word_tokenize` handles edge cases like contractions: "don't" → ["do", "n't"]

```
import nltk
nltk.download('punkt_tab', quiet=True)
sentence = "I don't love spam emails, do you?"
# Method 1: basic split (misses contractions)
basic = sentence.split()
print('Basic split:', basic)
```

```
# Method 2: NLTK's smarter tokeniser
from nltk.tokenize import word_tokenize
smart = word_tokenize(sentence.lower())
print('NLTK tokenize:', smart)
```

```
Basic split: ['I', "don't", 'love', 'spam', 'emails,', 'do', 'you?']
NLTK tokenize: ['i', 'do', "n't", 'love', 'spam', 'emails', ',', 'do', 'you', '?']
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

Step 5 — Stemming vs Lemmatisation

'Running', 'runs', 'ran', 'runner' all refer to the same core concept. We need to reduce them to a single base form:

Technique	Rule-based?	'running' →	'better' →	'studies' →	Best for
Stemming	No — chops suffix	run	bett (wrong!)	studi	Speed; BoW models
Lemmatisation	Yes — uses grammar	run	good (correct)	study	Quality; NLP pipelines

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
nltk.download('wordnet', quiet=True)
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
words = ['running', 'studies', 'better', 'caring', 'happily']
for word in words:
    stem = stemmer.stem(word)
    lemma = lemmatizer.lemmatize(word, pos='v')
    print(f'{word:12} | stem: {stem:10} | lemma: {lemma}')
```

```
running | stem: run | lemma: run
studies | stem: studi | lemma: study
better | stem: better | lemma: better
caring | stem: care | lemma: care
happily | stem: happili | lemma: happily
```

Complete pipeline in one function

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download(['stopwords', 'wordnet', 'punkt'], quiet=True)
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    # Step 1: Convert to lowercase
    text = text.lower()
    # Step 2: Remove punctuation and special characters
    text = re.sub(r'^a-z ', '', text)
    # Step 3: Tokenize
    tokens = text.split()
    # Step 4: Remove stopwords
    filtered_tokens = []
    for w in tokens:
        if w not in stop_words:
            filtered_tokens.append(w)

    tokens = filtered_tokens
    # Step 5: Lemmatize
    lemmatized_tokens = []
```

```

for w in tokens:
    lemmatized_tokens.append(
        lemmatizer.lemmatize(w)
    )
tokens = lemmatized_tokens
# Convert list back to string
return ' '.join(tokens)

# Test
print(preprocess('I am running to the stores to buy some groceries!'))

running store buy grocery

```

NOTE

Always lowercase BEFORE removing punctuation. Otherwise 'Hello!' and 'hello!' become different tokens after punctuation removal if you lowercase second. And always remove stopwords AFTER tokenisation — you need the word list first

Turning Text into Numbers

Computers only understand numbers — Bag of Words and TF-IDF convert text. After pre-processing you have clean words. But an ML model needs numbers. This section shows the two most important methods to convert a bag of words into a numeric matrix that any classifier can use.

✦ Bag of Words (BoW) — just count words

Imagine you have 3 emails and a vocabulary of 5 words. BoW simply counts how many times each word appears in each email and builds a table:

Email	cricket	match	free	win	money
Email 1 (cricket fan)	3	2	0	0	0
Email 2 (spam)	0	0	2	3	2
Email 3 (cricket spam)	2	1	1	2	1

```

from sklearn.feature_extraction.text import CountVectorizer
emails = [
    'cricket match cricket match cricket', # genuine fan
    'free win money free win money win', # spam
    'cricket match free win money cricket', # cricket spam
]
# CountVectorizer builds the vocab and counts automatically
cv = CountVectorizer()
X = cv.fit_transform(emails).toarray() # .toarray() makes it readable
print('Vocabulary:', cv.get_feature_names_out())
print('BoW matrix:')
print(X)

```

```

Vocabulary: ['cricket' 'free' 'match' 'money' 'win']
BoW matrix:
[[3 0 2 0 0]
 [0 2 0 2 3]
 [2 1 1 1 1]]

```

✦ TF-IDF — reward rare, important words

BoW treats 'the' and 'cricket' equally if they appear the same number of times. But 'the' is everywhere (useless) while 'cricket' in a spam email is very informative. TF-IDF fixes this by multiplying term frequency by how rare the word is across all documents:

TF-IDF = Term Frequency x Inverse Document Frequency

TF (Term Frequency)
How often a word appears
in THIS document

IDF (Inverse Doc Freq)
Rare words get HIGH weight
"cricket" rare -> IDF high

X

"cricket" appears 5 times -> TF=5

"the" everywhere -> IDF~0

```
from sklearn.feature_extraction.text import TfidfVectorizer
emails = [
    'cricket match cricket match cricket',
    'free win money free win money win',
    'cricket match free win money cricket',
]
tfidf = TfidfVectorizer(sublinear_tf=True, max_df=0.95, min_df=1)
X_tfidf = tfidf.fit_transform(emails).toarray()
print('Feature names:', tfidf.get_feature_names_out())
print('TF-IDF matrix (rounded):')
import numpy as np
print(np.round(X_tfidf, 3))
```

```
Feature names: ['cricket' 'free' 'match' 'money' 'win']
TF-IDF matrix (rounded):
[[0.778 0.    0.628 0.    0.   ]
 [0.    0.532 0.    0.532 0.659]
 [0.646 0.382 0.382 0.382 0.382]]
```

BoW vs TF-IDF — when to use which?

Use BoW when: your vocabulary is small, you need speed, or you're building a quick prototype. Use TF-IDF when: some words are much more informative than others (spam detection, document classification, search ranking). TF-IDF almost always beats plain BoW.